

Errors and Exceptions

Topics in this section:

1. Errors – the programmer’s daily bread
2. Natural languages and communication errors
3. Errors vs exceptions
4. Errors without exceptions
5. Limited confidence

Errors and Exceptions

Errors are an unavoidable part of programming. From minor typos to design flaws, errors can lead to unexpected program behaviors, crashes, or incorrect results. This is well summarized by Edsger W. Dijkstra:

"If debugging is the process of removing software bugs, then programming must be the process of putting them in."

Errors occur due to:

- **Misconceptions:** Incorrect problem-solving approaches.
- **Language misuse:** Incorrect syntax or vocabulary usage.
- **Unpredictable user behavior:** Scenarios developers fail to anticipate.

Programmers must minimize errors and mitigate their impact while acknowledging their inevitability.

Natural Languages and Communication Errors

Programming languages, like natural languages, rely on:

- **Syntax:** Rules governing sentence structure.
- **Vocabulary:** The defined set of words a language recognizes.

Errors can occur due to:

1. **Syntax Errors:** Violations of language rules (e.g., missing punctuation).
 - Example:

javascript

Copy code

```
let x = 5 // Missing semicolon (formal error)
```

2. **Semantic Errors:** Using invalid vocabulary or references.

- Example:

javascript

Copy code

```
console.log('msg'); // Correct
```

```
consolee.log('msg'); // ReferenceError: consolee is not defined
```

3. **Logical Errors:** Flawed logic in code that produces incorrect outcomes.

- Example:

javascript

Copy code

```
let add = (a, b) => a * b; // Incorrect logic; intended addition
```

ReferenceError in JavaScript

Occurs when trying to access an undefined function or variable:

- Example:

javascript

Copy code

```
let result = undefinedVar; // ReferenceError: undefinedVar is not defined
```

Errors like these are **semantic errors** and are detected **at runtime**.

Error Categories and Examples

1. **Syntax Errors:** Detected before code execution.

Example:

javascript

Copy code

```
let multiply = (a b) => a + b; // Missing comma between parameters
```

2. **Semantic Errors:** Detected during execution.

Example:

javascript

Copy code

```
let res = mult(5, 10); // ReferenceError: mult is not defined
```

3. **Logical Errors:** Cannot be detected automatically; require manual debugging.

Example:

javascript

Copy code

```
let multiply = (a, b) => a + b; // Adds instead of multiplying
```

JavaScript Error Handling

JavaScript handles runtime errors (exceptions) using the try...catch block:

- Example:

javascript

Copy code

```
try {  
  console.log('Hello');  
  console.log('World'); // ReferenceError  
} catch (error) {  
  console.log(error.message); // Output: console.log is not defined  
}
```

Benefits of try...catch:

- Prevents program crashes.
 - Provides meaningful error messages.
-

Errors Without Exceptions

Not all erroneous situations throw exceptions. For instance, attempting to access a non-existent property on an object returns undefined without an exception:

- Example:

javascript

Copy code

```
let obj = {};
```

```
console.log(obj.name); // undefined (no error)
```